

City Hall Operational Case Study

Revised library edition: 2026-08-20

Original case date: 2026-07-22

Workspace: Aptlantis Development Drive

Active standards library: D:\.library\aptlantis_core

Standards workshop and lineage: D:\.city_hall

Project: render-manifest.crate

Standard produced: Library Development Standard (LDS)

Executive Summary

This case study records a successful end-to-end Aptlantis governance workflow. A rough Rust library idea entered the workspace, was classified, received proposal records, exposed a missing library-specific standard, produced LDS through the standards-development process, and stopped before unauthorized implementation.

The original 2026-07-22 case framed City Hall as the governance center because that was the working model at the time. This revised library edition preserves the case evidence while updating the authority model:

- D:\.library\aptlantis_core is the active standards library and the home for solid/adopted governance material.
- D:\.city_hall is the standards workshop, sandbox, promotion path, and historical lineage store.
- City Hall-only drafts do not govern active projects until promoted into aptlantis_core or explicitly adopted by a governing standard.

The practical lesson remains the same: governance is useful when it lets a future human or agent recover context, classify work correctly, avoid premature implementation, and leave a durable handoff.

What Happened

A loosely formed idea for a Rust crate was introduced into the Aptlantis development workspace. The intended capability was a library that could import structured TOML records and render them into sections, properties, tables, text, and evidence.

The idea was useful but incomplete. At intake time:

- no source code existed;
- crate names were not final;
- the public API was speculative;
- the governing delivery standard had not been identified;
- the project had not reached implementation readiness.

The project was therefore treated as a proposal, not as an implementation task.

Governance Path

The agent entered through documented workspace governance rather than relying on conversation memory. It used the read-first path to recover:

- the root workspace instructions;
- the project manifest and proposal material;
- the project README and library interface note;
- the active governing standards;
- the standards-development process for missing standard coverage.

Under the current authority model, the active read path begins with D:\Development.manifest.toml, D:\AGENTS.md, D:\INDEX.md, and D:\library\aptlantis_core. City Hall remains the right place to inspect lineage, draft standards, review notes, and promotion history.

Classification Decision

The agent classified the proposed work as a library-first Rust crate family. That distinction mattered because the primary consumer would be other code, not a human launching a desktop app, running a CLI, deploying a website, or preserving a dataset.

The existing delivery standards did not fit cleanly:

- DRS governed desktop applications.
- CTS governed command tools.
- WDS governed websites and web applications.
- DDS governed datasets.
- SIS governed services and infrastructure.

The agent correctly rejected reusing DDS for a library merely because the library would read structured records. Data input did not make the project a dataset.

Standard Gap And LDS

Because no active standard governed library-first packages, the workflow exposed a real standards gap. The agent followed SFDS to create the Library Development Standard.

LDS defined the expectations for libraries, crates, packages, SDKs, and code-consumed modules:

- public API boundaries;
- stability and versioning expectations;
- compatibility and extension contracts;
- documentation for consumers;
- relationship to companion CLI or service surfaces;
- evidence needed before claiming release readiness.

The planned crate family could then be governed honestly:

- library surfaces by LDS;
- CLI surfaces by CTS;
- service surfaces by SIS when applicable.

Controlled Stop

The agent stopped before implementation. That was not a failure of productivity; it was the expected governance behavior.

The project was still at proposal readiness. Creating source code, scaffolding a Cargo workspace, or inventing public APIs would have skipped the documented lifecycle gate.

The durable output was therefore:

- a classified project proposal;
- a documented standard gap;
- LDS as a new standard;
- standard assignments for planned surfaces;
- a handoff that preserved next safe actions.

Why The Case Belongs In aptlantis_core

The case is no longer merely City Hall workshop material. It is adopted evidence for how the active governance system should behave.

For that reason, the active copy belongs beside the standards it explains:

```
D:\.library\aptlantis_core\City Hall Operational Case Study.pdf
```

City Hall may keep historical copies or references, but active readers should find this case through the library.

Current Recovery Notes

As of this revised edition:

- D:\Development.manifest.toml has been restored as the machine-readable root registry.
- D:\.library\aptlantis_core\README.md and D:\.library\aptlantis_core\WORKSHOP-MAP.md are the active overview entry points.
- D:\.city_hall\README.md and D:\.city_hall\WORKSHOP-MAP.md are workshop and lineage entry points.
- Release hashing is governed by ARHS in aptlantis_core.
- Archive preservation hashing and detached archive signatures are governed by AAMHS.

Outcome

The case demonstrates that Aptlantis governance can:

- orient an agent with limited prior context;
- distinguish project intent from implementation temptation;
- identify the right governing standard;
- create a missing standard through the standards framework;
- assign mixed standards to mixed delivery surfaces;
- preserve evidence and handoff notes;
- stop when the lifecycle state says implementation is premature.

That is the value of the system. It does not exist to multiply documents. It exists so work can survive time, ambiguity, and handoff.